

Buffer Overflow

8

CS/COE 0449

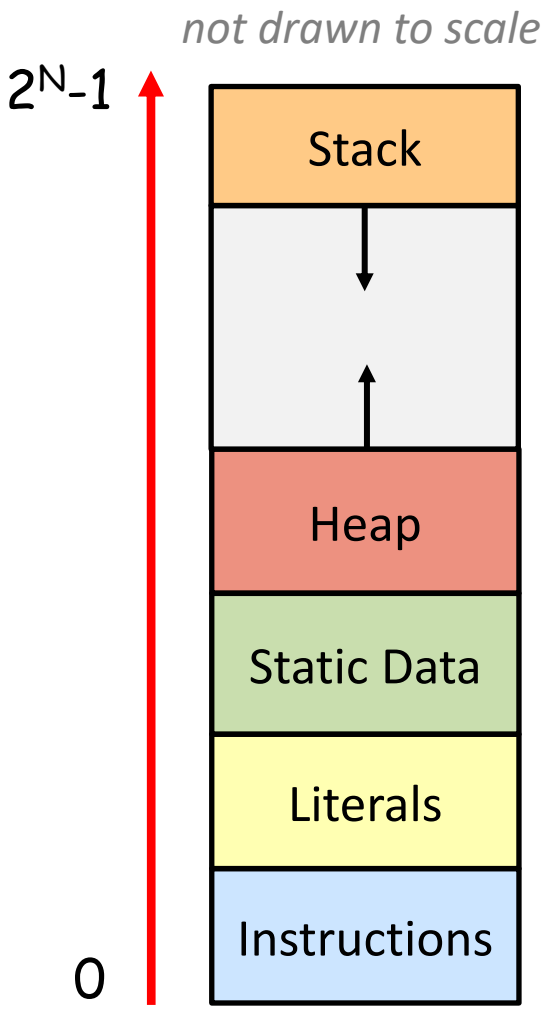
Introduction to Systems Software

Luis Oliveira

(with content borrowed from wilkie and Vinicius Petrucci)

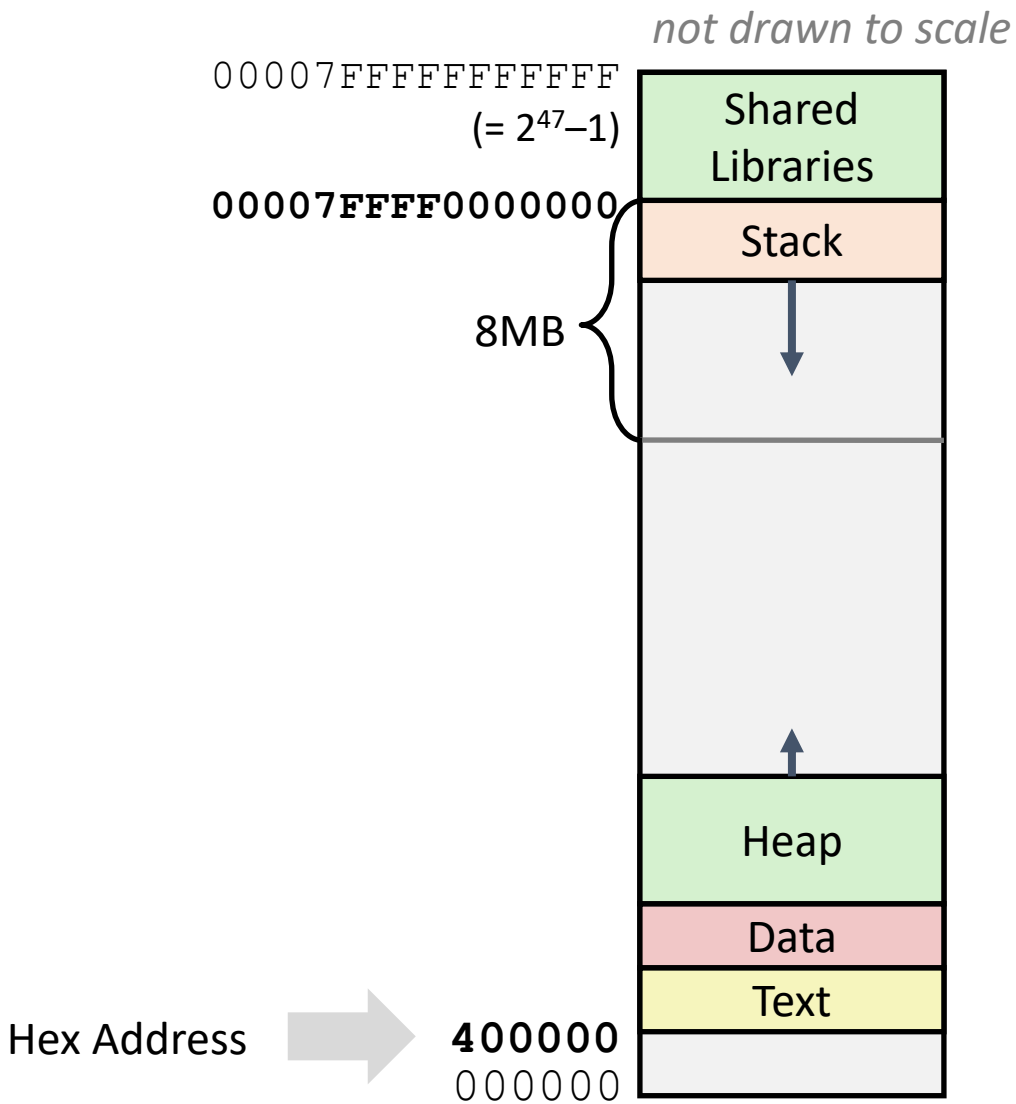
Review: General Memory Layout

- Stack
  - Local variables (procedure context)
- Heap
  - Dynamically allocated as needed
  - `malloc(),calloc(),new(),…`
- Statically allocated Data
  - Read/Write: global variables
  - Read-only: string literals
- Code/Instructions
  - Executable machine instructions
  - Read-only



x86-64 Linux Memory Layout

- Stack
  - Runtime stack (8MB limit)
  - e.g., local variables
- Heap
  - Dynamically allocated as needed
  - When call `malloc()`, `new()`
- Data
  - Statically allocated data
  - e.g., global/static vars, string constants
- Text / Shared Libraries
  - Executable machine instructions
  - Read-only



Memory Allocation Example

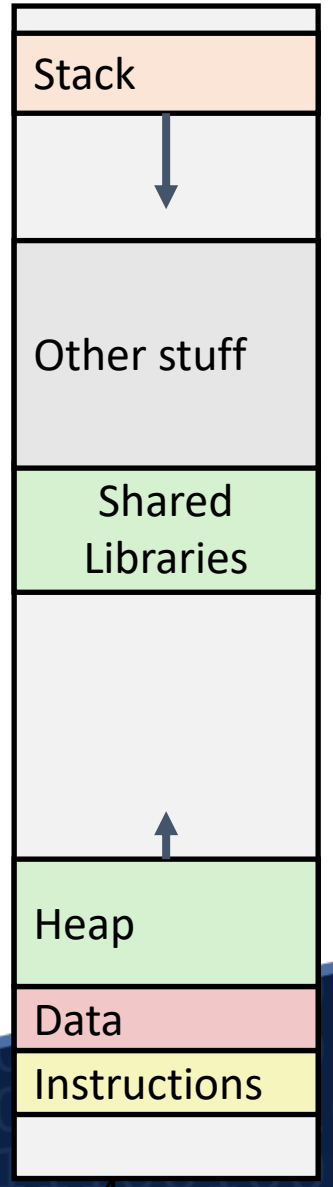
not drawn to scale

```
char big_array[1L<<24];  /* 16 MB */
char huge_array[1L<<31]; /*  2 GB */

int global = 0;

int useless() { return 0; }

int main()
{
    void *p1, *p2, *p3, *p4;
    int local = 0;
    p1 = malloc(1L << 8); /* 256 B */
    p2 = malloc(1L << 8);  /* 256  B */
    p3 = malloc(1L << 8); /*  256 B */
    p4 = malloc(1L << 8);  /* 256  B */
    /* Some print statements ... */
}
```



Where does everything go?

Memory Allocation Example

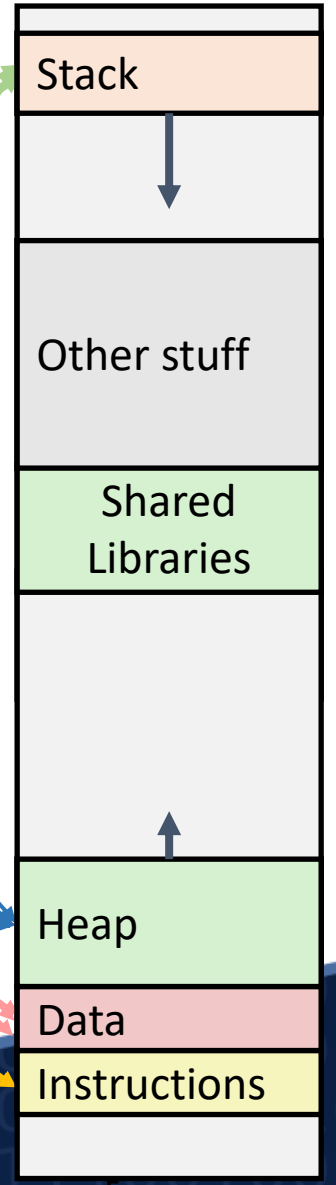
not drawn to scale

```
char big_array[1L<<24];  /* 16 MB */
char huge_array[1L<<31]; /*  2 GB */

int global = 0;

int useless() { return 0; }

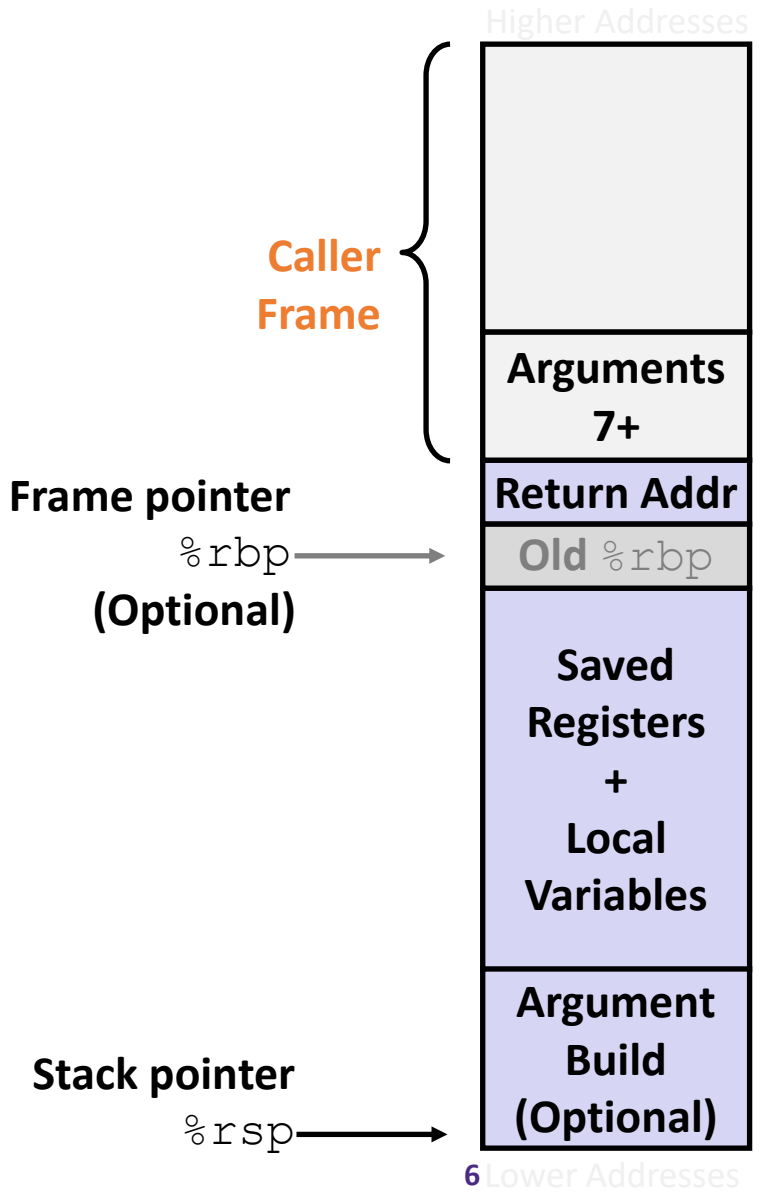
int main()
{
    void *p1, *p2, *p3, *p4;
    int local = 0;
    p1 = malloc(1L << 8); /* 256 B */
    p2 = malloc(1L << 8); /* 256 B */
    p3 = malloc(1L << 8); /* 256 B */
    p4 = malloc(1L << 8); /* 256 B */
    /* Some print statements ... */
}
```



Where does everything go?

Reminder: x86-64/Linux Stack Frame

- Caller's Stack Frame
  - Arguments (if > 6 args) for this call
- Current/Callee Stack Frame
  - Return address
    - Pushed by `call` instruction
  - Old frame pointer (optional)
  - Saved register context (when reusing registers)
  - Local variables (if can't be kept in registers)
  - "Argument build" area (If callee needs to call another function - parameters for function about to call, if needed)



Recall: Memory Referencing Bug Example

```
typedef struct {
  int a[2];
  double d;
} struct_t;

double fun(int i) {
  struct_t s;
  s.d = 3.14;
  s.a[i] = 1073741824; /* Possibly out of bounds */
  return s.d;
}
```

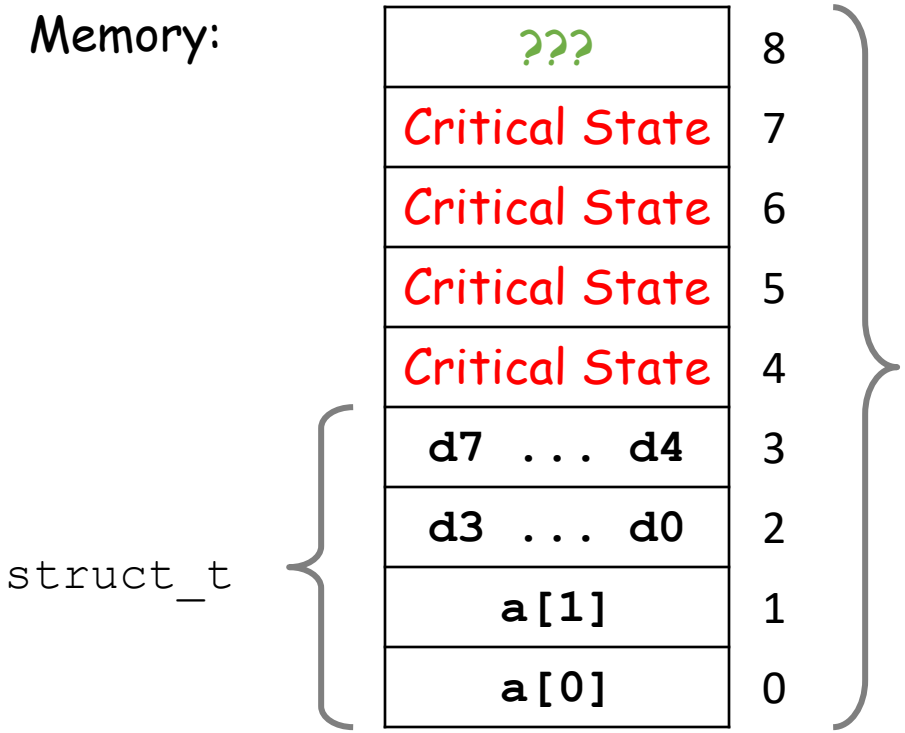
```
fun(0)  ->3.1400000000
fun(1)  ->3.1400000000
fun(2)  ->3.1399998665
fun(3)  ->2.0000006104
fun(4)  ->Segmentation fault
```

Result is system specific

Memory Referencing Bug Example

```
typedef struct {
  int a[2];
  double d;
} struct_t;
```

```
fun(0)   ->3.1400000000
fun(1)   ->3.1400000000
fun(2)   ->3.1399998665
fun(3)   ->2.0000006104
fun(4)   ->Segmentation fault
fun(8)   ->3.1400000000
```



Buffer Overflow

- Traditional Linux memory layout provide opportunities for malicious programs
  - Stack grows “backwards” in memory
  - Data and instructions both stored in the same memory

- Recall that C does not check array bounds
  - Many Unix/Linux/C functions don’t check argument sizes
  - Allows overflowing (writing past the end) of buffers (arrays)

Buffer Overflow (cont.)

- Buffer overflows on the stack can overwrite “interesting” data
  - Attackers just choose the right inputs
- Simplest form (sometimes called “stack smashing”)
  - Unchecked length on string input into bounded array causes overwriting of stack data
  - Try to change the return address of the current procedure
- Why is this a big deal?
  - It is (was?) the #1 technical cause of security vulnerabilities
    - #1 overall cause is social engineering / user ignorance

String Library Code

- Implementation of Unix function `gets()`

```
/* Get string from stdin */
char* gets(char* dest) {
    int c = getchar();
    char* p = dest;
    while (c != EOF && c != '\n') {
        *p++ = c;
        c = getchar();
    }
    *p = '\0';
    return dest;
}
```

pointer to start of an array

same as:

```
*p = c;
p++;
```

What could go wrong in this code?

String Library Code

- Implementation of Unix function gets()

```
/* Get string from stdin */
char* gets(char* dest) {
    int c = getchar();
    char* p = dest;
    while (c != EOF && c != '\n') {
        *p++ = c;
        c = getchar();
    }
    *p = '\0';
    return dest;
}
```

```
GETS(3)                Linux Programmer's Manual                GETS(3)

NAME
       gets - get a string from standard input (DEPRECATED)

BUGS
       Never use gets().  Because it is impossible to tell without
       knowing the data in advance how many characters gets() will
       read, and because gets() will continue to store characters
       past the end of the buffer, it is extremely dangerous to
       use.  It has been used to break computer security.  Use
       fgets() instead.

       For more information, see CWE-242 (aka "Use of Inherently
       Dangerous Function") at http://cwe.mitre.org/data/defini-
       tions/242.html
```

- No way to specify limit on number of characters to read
- Similar problems with other Unix functions:
  - `strcpy`: Copies string of arbitrary length to a `dst`
  - `scanf`, `fscanf`, `sscanf`, when given `%s` specifier

Vulnerable Buffer Code

Code example to try on Thoth!
https://bit.ly/3eWwndQ

```
/* Echo Line */
void echo() {
    char buf[8];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

← BTW, how big is big enough?

```
void call_echo() {
    echo();
}
```

```
unix> ./buf-nsp
Enter string: 12345678901234567890123
12345678901234567890123
```

```
unix> ./buf-nsp
Enter string: 123456789012345678901234
Segmentation Fault
```

Buffer Overflow Disassembly

echo:

24 bytes (decimal)

```
000000000040069c <echo>:
  40069c:   48 83 ec 18           sub    $0x18,%rsp
  4006a0:   48 89 e7              mov    %rsp,%rdi
  4006a3:   e8 a5 ff ff ff        callq  40064d <gets>
  4006a8:   48 89 e7              mov    %rsp,%rdi
  4006ab:   e8 50 fe ff ff        callq  400500 <puts@plt>
  4006b0:   48 83 c4 18           add    $0x18,%rsp
  4006b4:   c3                    retq
```

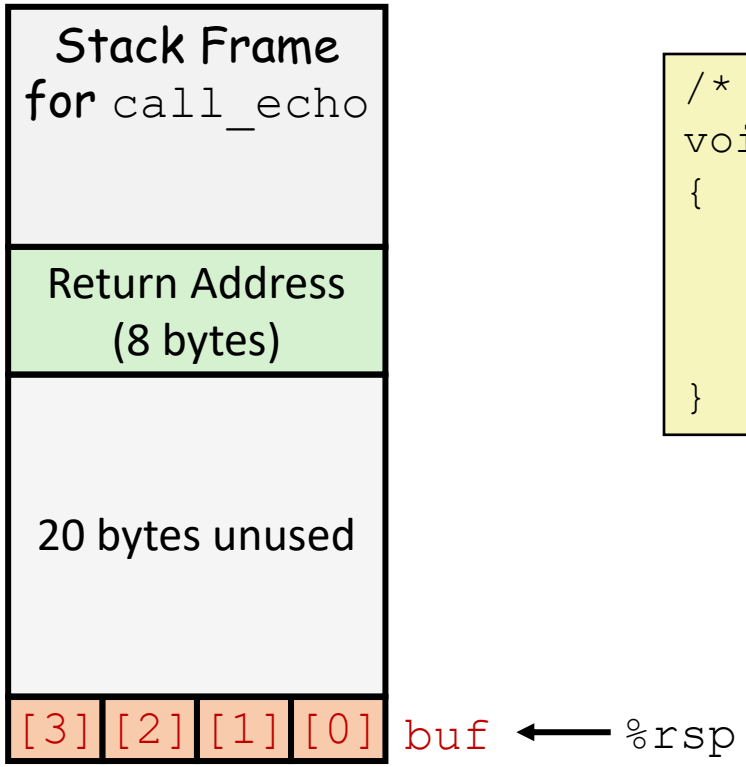
call_echo:

return address

```
  4006b5:   48 83 ec 08           sub    $0x8,%rsp
  4006b9:   b8 00 00 00 00        mov    $0x0,%eax
  4006be:   e8 d9 ff ff ff        callq  40069c <echo>
  4006c3:   48 83 c4 08           add    $0x8,%rsp
  4006c7:   c3                    retq
```

Buffer Overflow Stack Example

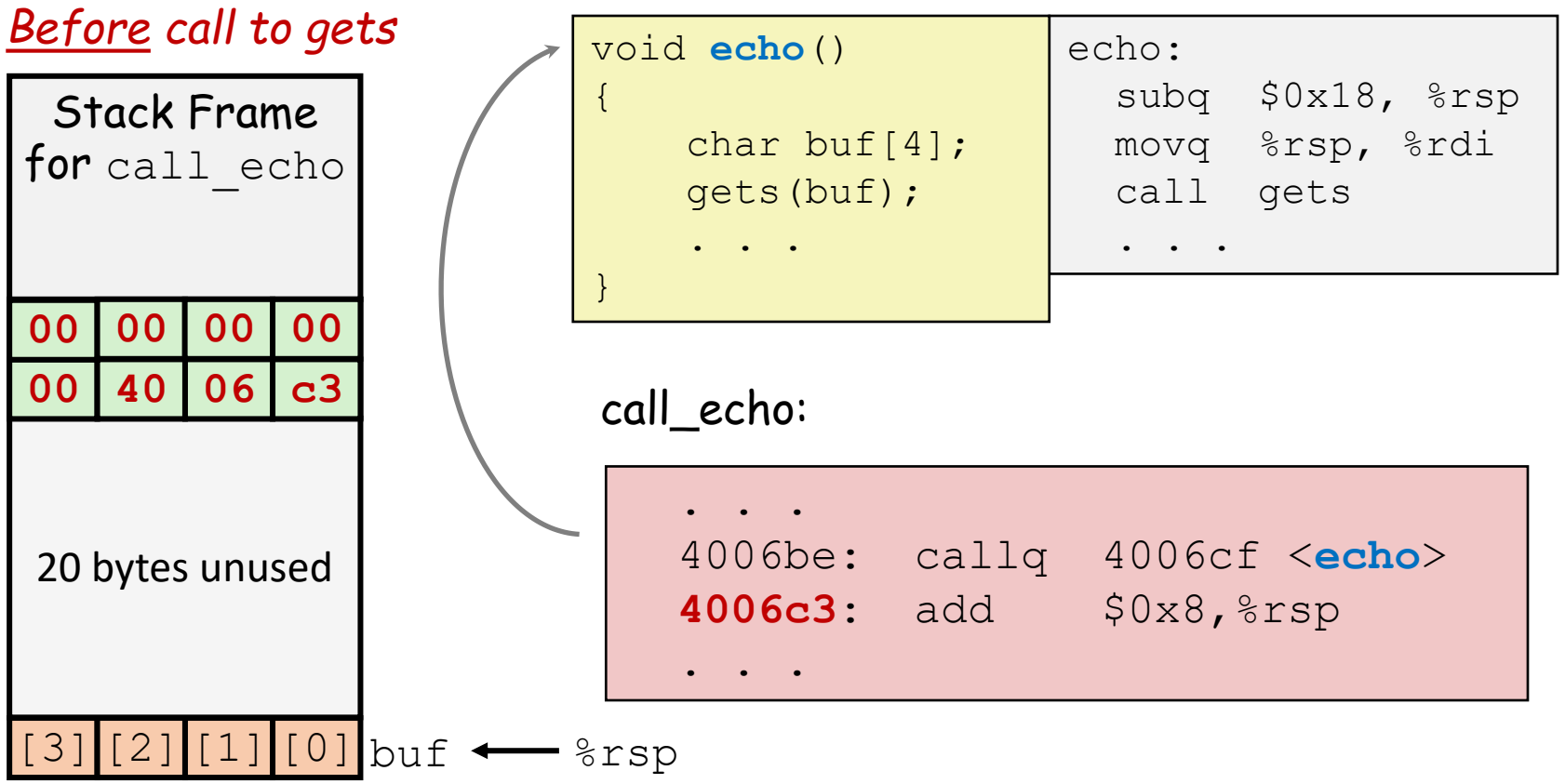
Before call to gets




```
/* Echo Line */
void echo()
{
    char buf[4];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
  subq $0x18, %rsp
  movq  %rsp, %rdi
  call  gets
  . . .
```

Buffer Overflow Stack Example



Buffer Overflow Stack Example #1

After call to gets

Stack Frame for `call_echo`

00	00	00	00
00	40	06	c3
00	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ← %rsp

```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
   subq   $0x18, %rsp
   movq   %rsp, %rdi
   call   gets
   . . .
```

call_echo:

```
   . . .
   4006be:   callq   4006cf <echo>
   4006c3:   add     $0x8,%rsp
   . . .
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901
0123456789012345678901
```

"0123456789012345678901\0"

Overflowed buffer, but did not corrupt state

Buffer Overflow Stack Example #2

After call to gets

Stack Frame for `call_echo`

00	00	00	00
00	40	06	**00**
33	32	31	30
39	38	37	36
35	34	33	32
31	30	39	38
37	36	35	34
33	32	31	30

buf ← %rsp

```
void echo()
{
    char buf[4];
    gets(buf);
    . . .
}
```

```
echo:
   subq   $0x18, %rsp
   movq   %rsp, %rdi
   call   gets
   . . .
```

call_echo:

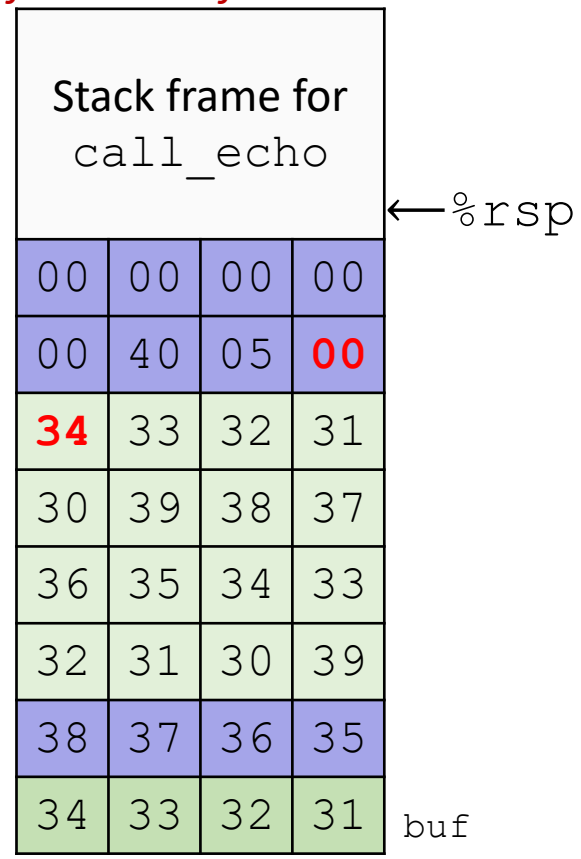
```
   . . .
   4006be:   callq   4006cf <echo>
   4006c3:   add     $0x8,%rsp
   . . .
```

```
unix>./bufdemo-nsp
Type a string:0123456789012345678901233
0123456789012345678901233
Segmentation fault
```

Program "returned" to 0x0400600, and then crashed.

Buffer Overflow Example #2 Explained

After return from echo




```
0000000000400500 <deregister_tm_clones>:
  400500:   mov    $0x60104f,%eax
  400505:   push   %rbp
  400506:   sub    $0x601048,%rax
  40050c:   cmp    $0xe,%rax
  400510:   mov    %rsp,%rbp
  400513:   jbe    400530
  400515:   mov    $0x0,%eax
  40051a:   test   %rax,%rax
  40051d:   je     400530
  40051f:   pop    %rbp
  400520:   mov    $0x601048,%edi
  400525:   jmpq   *%rax
  400527:   nopw   0x0(%rax,%rax,1)
  40052e:   nop
  400530:   pop    %rbp
  400531:   retq
```

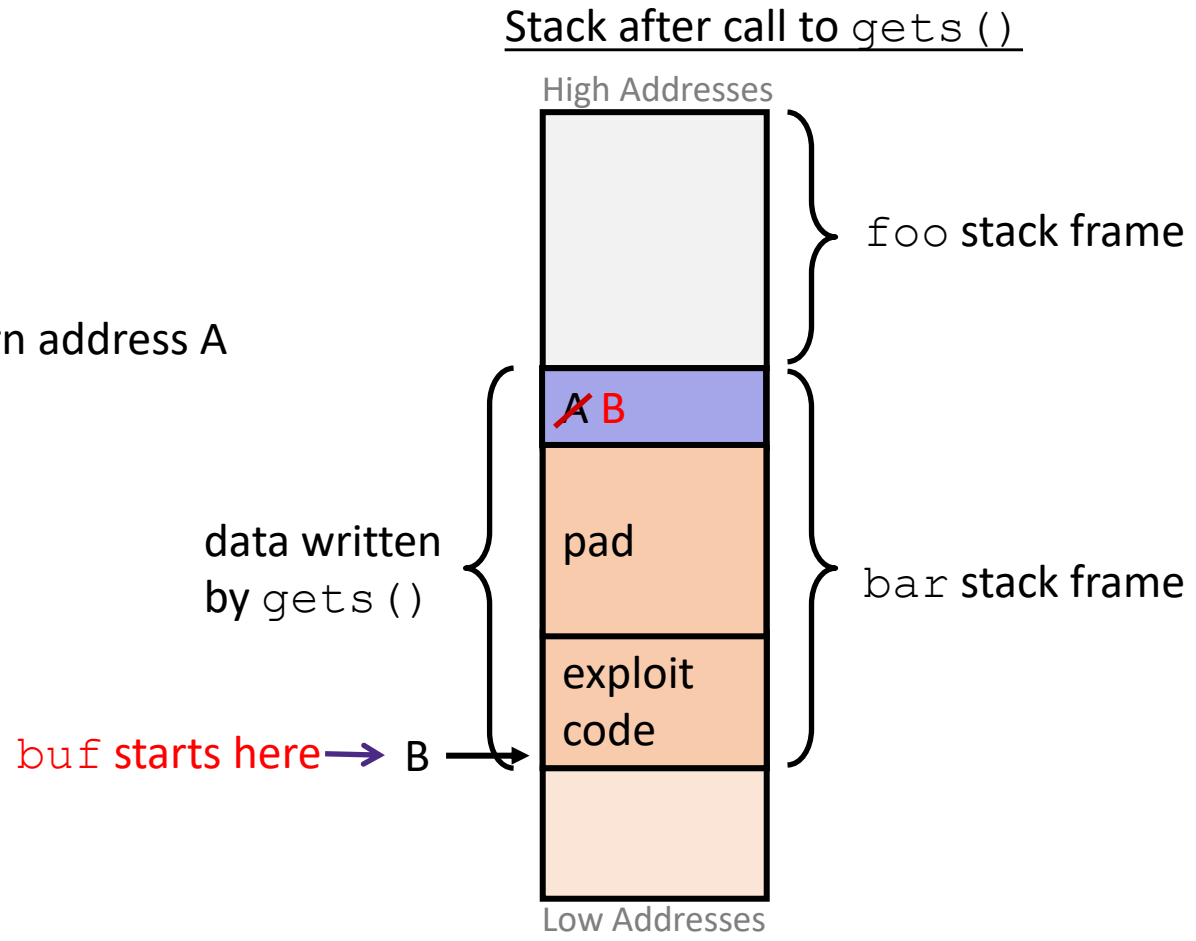
"Returns" to unrelated code, but continues!
Eventually segfaults on `retq` of `deregister_tm_clones`.

Malicious Use of Buffer Overflow: Code Injection Attacks

```
void foo(){
  bar();
A:...
}
```

return address A

```
int bar() {
  char buf[64];
  gets(buf);
  ...
  return ...;
}
```



- Input string contains byte representation of executable code
- Overwrite return address A with address of buffer B
- When `bar()` executes `ret`, will jump to exploit code

Exploits Based on Buffer Overflows

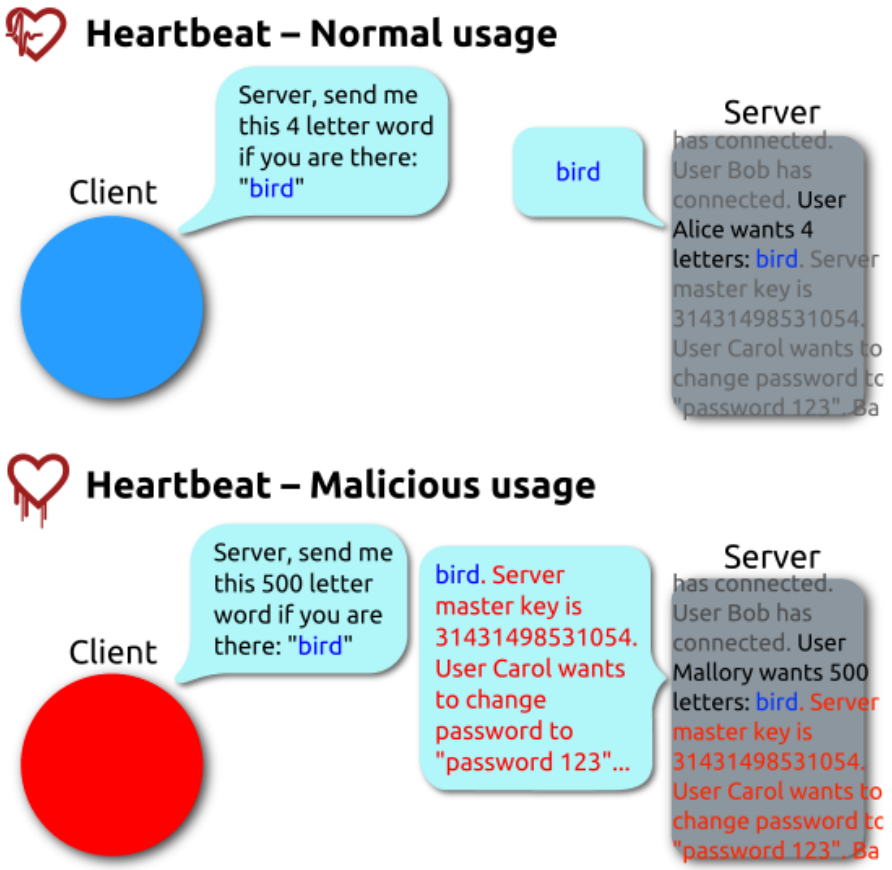
- *Buffer overflow bugs can allow remote machines to execute arbitrary code on victim machines*
- Distressingly common in real programs
  - Programmers keep making the same mistakes ☹
  - Recent measures make these attacks much more difficult
- Examples across the decades
  - Original "Internet worm" (1988)
  - *Still happens!!*
    - Heartbleed (2014, affected 17% of servers)
    - Cloudbleed (2017)
  - *Fun:* Nintendo hacks
    - Using glitches to rewrite code: https://www.youtube.com/watch?v=TqK-2jUQBUY
    - FlappyBird in Mario: https://www.youtube.com/watch?v=hB6eY73sLV0

Example: the original Internet worm (1988)

- Exploited a few vulnerabilities to spread
  - Early versions of the finger server (`fingerd`) used `gets()` to read the argument sent by the client:
    - `finger droh@cs.cmu.edu`
  - Worm attacked `fingerd` server with phony argument:
    - `finger "exploit-code padding new-return-addr"`
    - Exploit code: executed a root shell on the victim machine with a direct TCP connection to the attacker
- Scanned for other machines to attack
  - Invaded ~6000 computers in hours (10% of the Internet)
    - see June 1989 article in *Comm. of the ACM*
  - The young author of the worm was prosecuted…

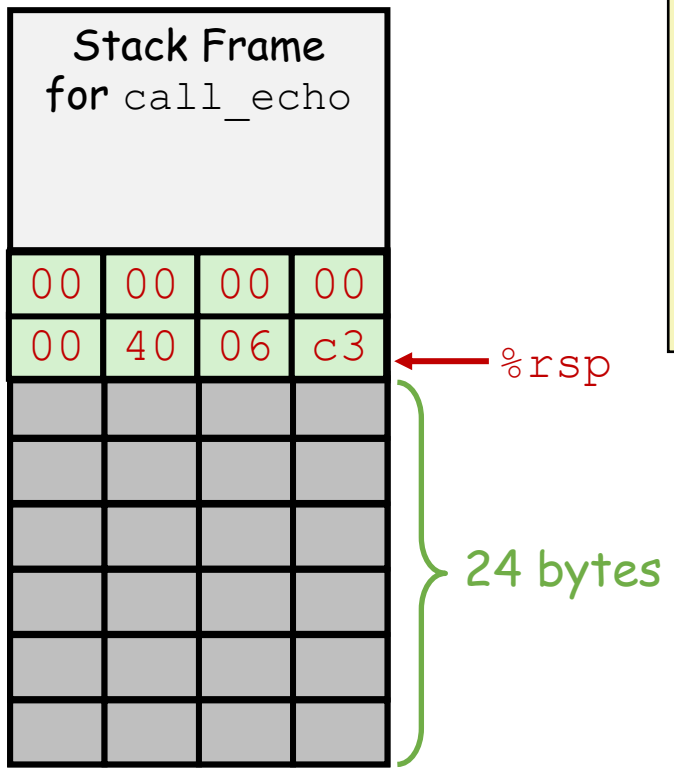
Heartbleed (2014)

- Buffer over-read in OpenSSL
  - Open source security library
  - Bug in a small range of versions
- “Heartbeat” packet
  - Specifies length of message
  - Server echoes it back
  - Library just “trusted” this length
  - Allowed attackers to read contents of memory anywhere they wanted
- Est. 17% of Internet affected
  - “Catastrophic”
  - Github, Yahoo, Stack Overflow, Amazon AWS, ...



By FenixFeather - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=32276981

Crafting Smashing String




```
int echo() {
  char buf[4];
  gets(buf);
  ...
  return ...;
}
```

Target Code

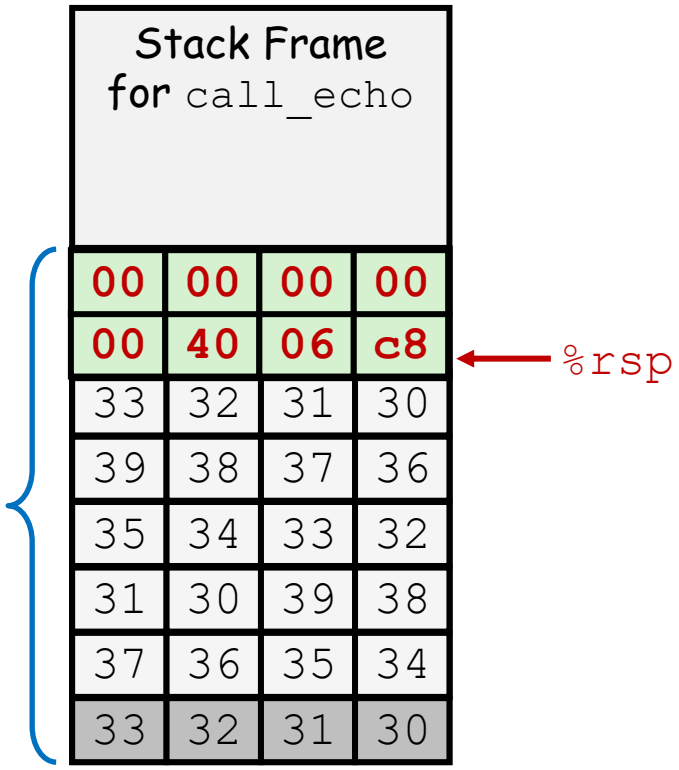
```
void smash() {
  printf("I've been smashed!\n");
  exit(0);
}
```

```
0000000004006c8 <smash>:
  4006c8:       48 83 ec 08
```

Attack String (Hex)

```
30 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 37 38 39 30 31 32 33
c8 06 40 00 00 00 00 00
```

Smashing String Effect



Attack String (Hex)

```
30 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 37 38 39 30 31 32 33
c8 06 40 00 00 00 00 00
```

Target Code

```
void smash() {
  printf("I've been smashed!\n");
  exit(0);
}
```

```
00000000004006c8 <smash>:
  4006c8:       48 83 ec 08
```

Performing Stack Smash

Code example to try on Thoth!
https://bit.ly/3eWwndQ

- Put hex sequence in file smash-hex.txt
- Use `hexify` program to convert hex digits to characters
  - Some of them are non-printing
- Provide as input to vulnerable program

```
$ cat smash-hex.txt
30 31 32 33 34 35 36 37 38 39 30 31 32 33 34 35 36 37 38 39 30 31 32 33
c8 06 40 00 00 00 00 00
$ cat smash-hex.txt | ./hexify | ./bufdemo-nsp
Type a string:012345678901234567890123?@
I've been smashed!
```

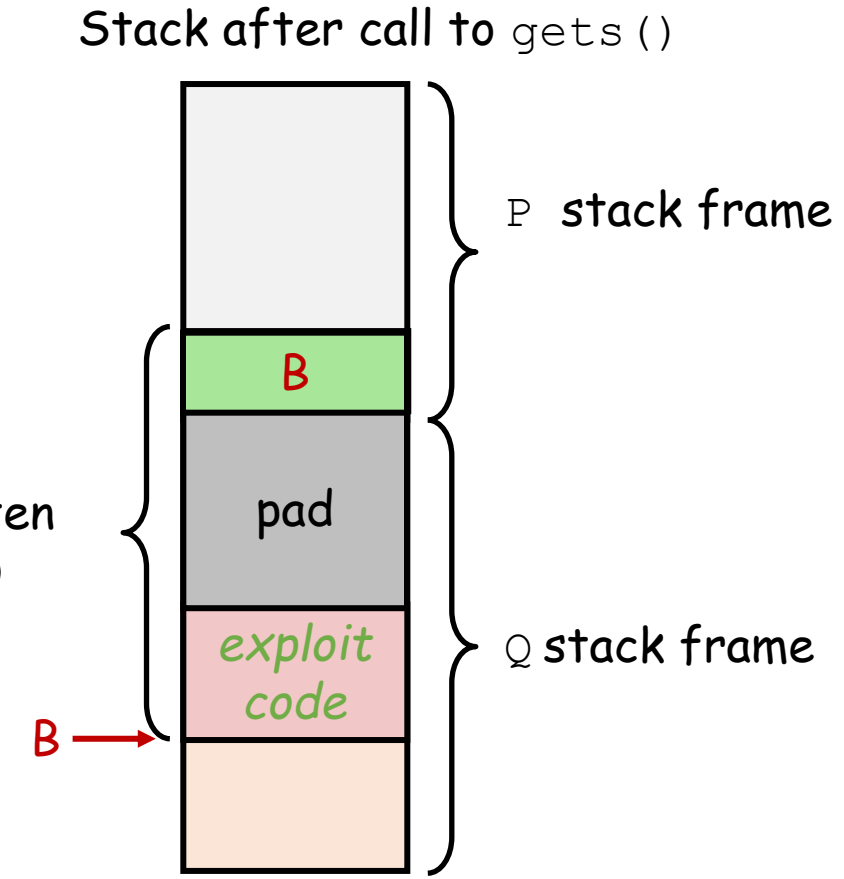
```
void smash() {
  printf("I've been smashed!\n");
  exit(0);
}
```

Code Injection Attacks

```
void P(){
  Q();
A:  ...
}
```

Return address *A*

```
int Q() {
  char buf[64];
  gets(buf);
  ...
  return ...;
}
```

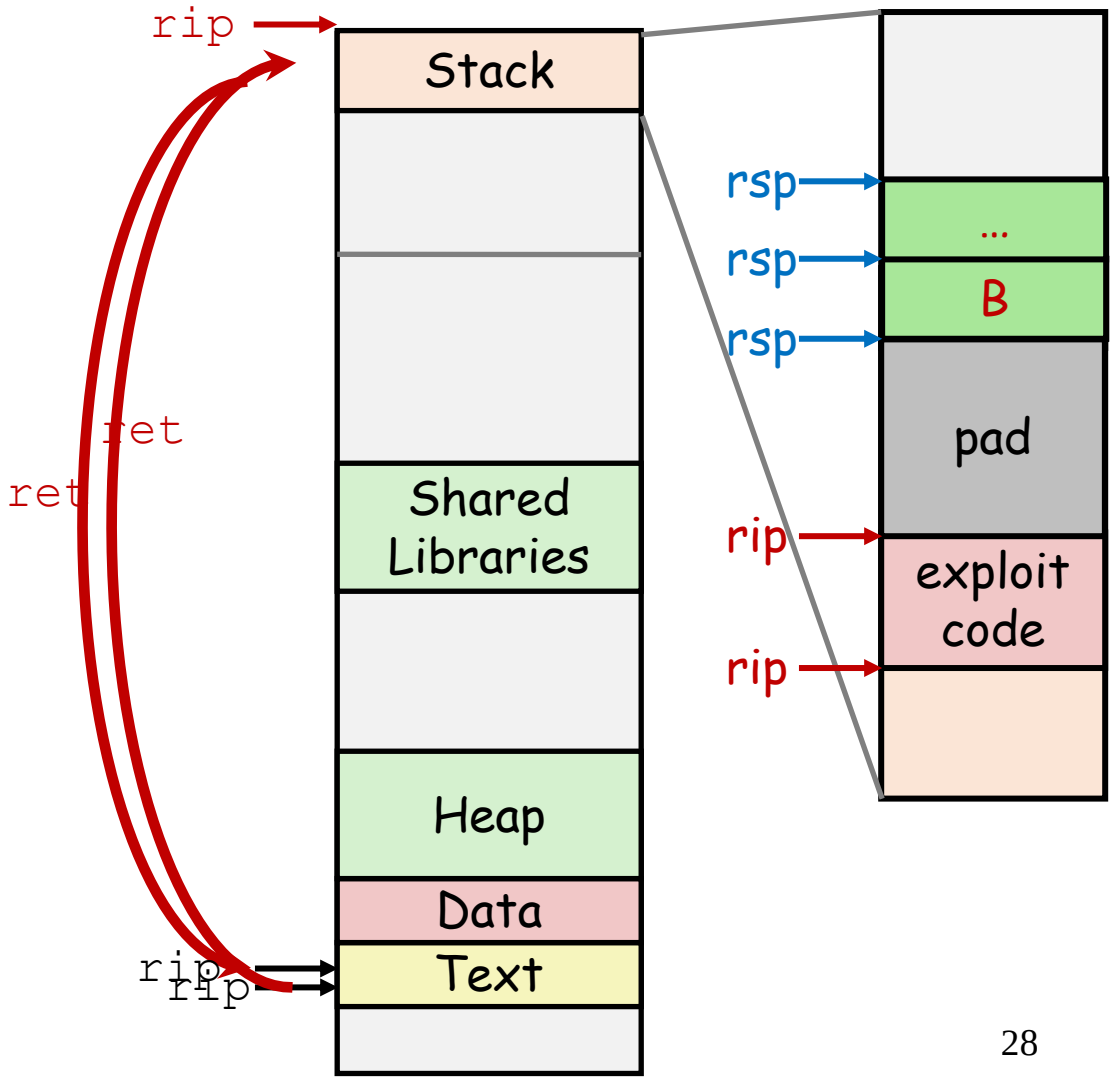


- Input string contains byte representation of executable code
- Overwrite return address A with address of buffer B
- When Q executes `ret`, will jump to exploit code

How Does The Attack Code Execute?

```
void P(){
  Q();
  ...
}
```

```
int Q() {
  char buf[64];
  gets(buf); // A->B
  ...
  return ...;
}
```



Dealing with buffer overflow attacks

1) Avoid overflow vulnerabilities
2) Employ system-level protections
3) Have compiler use “stack canaries”

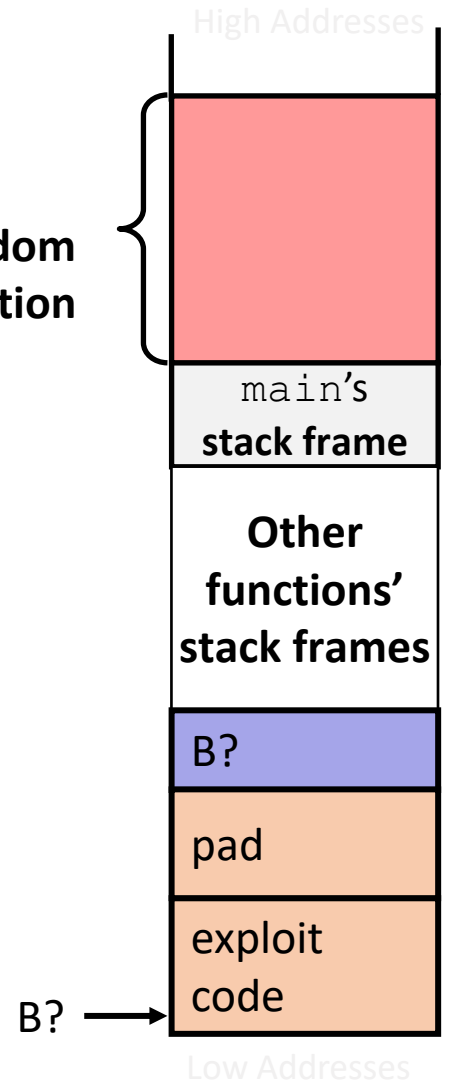
1) Avoid Overflow Vulnerabilities in Code

- Use library routines that limit string lengths
  - `fgets` instead of `gets` (2nd argument to `fgets` sets limit)
  - `strncpy` instead of `strcpy`
  - Don't use `scanf` with `%s` conversion specification
    - Use `fgets` to read the string
    - Or use `%ns` where `n` is a suitable integer

```
/* Echo Line */
void echo()
{
    char buf[8];  /* Way too small! */
    fgets(buf, 8, stdin);
    puts(buf);
}
```

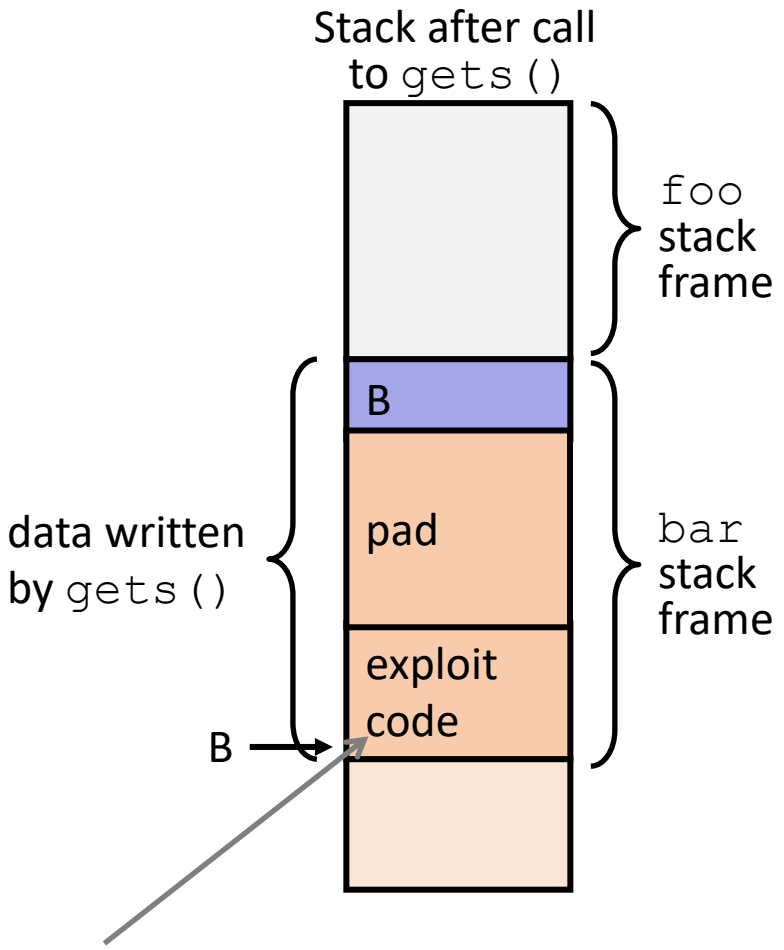
2) System-Level Protections

- **Randomized stack offsets**
  - At start of program, allocate random amount of space on stack
  - Shifts stack addresses for entire program
    - Addresses will vary from one run to another
  - Makes it difficult for hacker to predict beginning of inserted code
- Example: Code from Slide 6 executed 5 times; address of variable `local` =
    - `0x7ffd19d3f8ac`
    - `0x7ffe8a462c2c`
    - `0x7ffe927c905c`
    - `0x7ffefd5c27dc`
    - `0x7fffa0175afc`
  - Stack repositioned each time program executes



2) System-Level Protections

- **Non-executable code segments**
  - In traditional x86, can mark region of memory as either “read-only” or “writeable”
    - Can execute anything readable
  - x86-64 added explicit “execute” permission
  - Stack marked as non-executable
    - Do *NOT* execute code in Stack, Static Data, or Heap regions
    - Hardware support needed



Any attempt to execute this code will fail

3) Stack Canaries

- Basic Idea: place special value ("canary") on stack just beyond buffer
  - *Secret* value known only to compiler
  - "After" buffer but before return address
  - Check for corruption before exiting function
- GCC implementation (now default)
  - `-fstack-protector`
  - Code back on Slide 14 (`buf-nsp`) compiled with `-fno-stack-protector` flag

```
unix>./buf
Enter string: 12345678
12345678
```

```
unix> ./buf
Enter string: 123456789
*** stack smashing detected ***
```

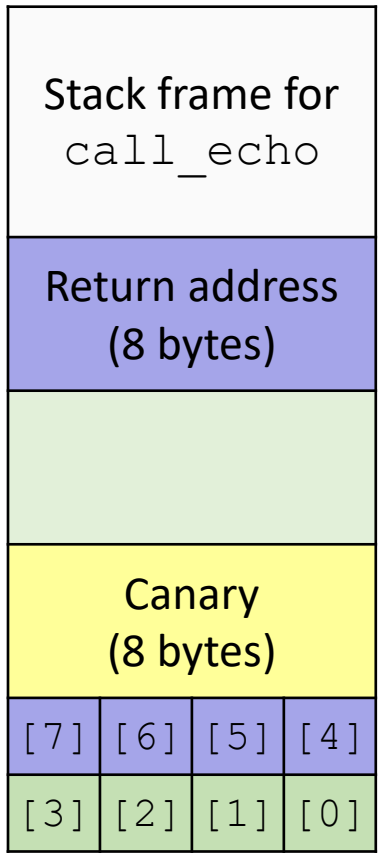
Protected Buffer Disassembly (buf)

echo:

```
400638:   sub    $0x18,%rsp
40063c:   mov    %fs:0x28,%rax
400645:   mov    %rax,0x8(%rsp)
40064a:   xor    %eax,%eax
 ...      ... call printf ...
400656:   mov    %rsp,%rdi
400659:   callq  400530 <gets@plt>
40065e:   mov    %rsp,%rdi
400661:   callq  4004e0 <puts@plt>
400666:   mov    0x8(%rsp),%rax
40066b:   xor    %fs:0x28,%rax
400674:   je     40067b <echo+0x43>
400676:   callq  4004f0 <__stack_chk_fail@plt>
40067b:   add    $0x18,%rsp
40067f:   retq
```

Setting Up Canary

Before call to gets



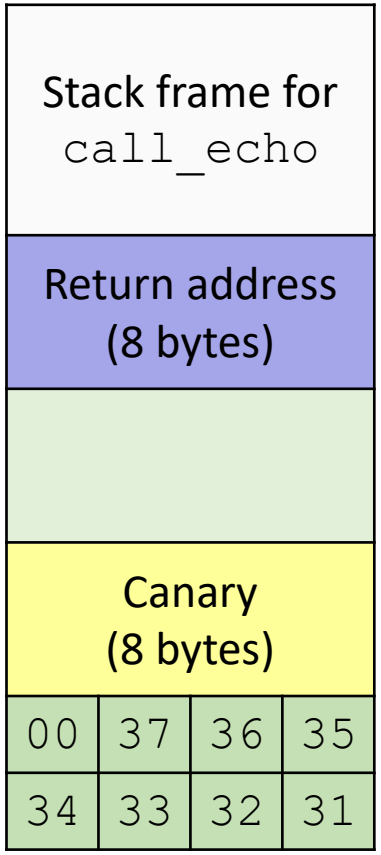

```
/* Echo Line */
void echo()
{
    char buf[8];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

Segment register
(don't worry about it)

```
echo:
    . . .
    movq    %fs:40, %rax    # Get canary
    movq    %rax, 8(%rsp)   # Place on stack
    xorl    %eax, %eax      # Erase canary
    . . .
```

Checking Canary

After call to gets




```
/* Echo Line */
void echo()
{
    char buf[8];  /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
    . . .
    movq    8(%rsp), %rax     # retrieve from Stack
    xorq    %fs:40, %rax      # compare to canary
    je      .L2               # if same, OK
    call    __stack_chk_fail  # else, FAIL
.L6:        . . .
```

Input: *1234567*

Summary

1) Avoid overflow vulnerabilities
   - Use library routines that limit string lengths

2) Employ system-level protections
   - Randomized Stack offsets
   - Code on the Stack is not executable

3) Have compiler use “stack canaries”